ÉTABLISSEMENT D'ENSEIGNEMENT SUPÉRIEUR

PROJET DE FIN D'ÉTUDES

# SOUK EL HAYAWAN

Conception et développement d'une plateforme web tunisienne de mise en relation autour des animaux

Vente • Adoption gratuite • Don • Entraide pour animaux disparus et trouvés

**Réalisé par**
Nom et prénom à renseigner

**Encadré par**
Encadrant à renseigner

**Année universitaire**
2026–2027 *(à confirmer)*

**État du document**
Avant développement – cadrage et conception

# Résumé

Ce rapport présente le cadrage et la conception progressive de **Souk El Hayawan**, une plateforme web destinée aux annonces d'animaux d'élevage et de compagnie en Tunisie. Le périmètre fonctionnel retenu comprend la **vente**, l'**adoption gratuite**, le **don**, ainsi qu'une rubrique distincte d'entraide pour les animaux **perdus, présumés volés ou trouvés**. La consultation s'appuie sur la catégorie, la race lorsqu'elle est pertinente, le prix pour les annonces tarifées et la localisation. Une mise en relation pour reproduction/saillie est envisagée, mais demeure **à confirmer**.

L'étude compare des plateformes existantes et identifie un positionnement fondé sur la structuration des données et la cohérence des parcours. L'architecture envisagée associe **Angular**, **Spring Boot**, **PostgreSQL**, **JPA** et **Flyway**, dans un monolithe modulaire avant une éventuelle évolution vers les microservices. Le backlog global est décomposé en sprints ; la méthode de conception comporte, pour chaque sprint, des cas d'utilisation, des descriptions détaillées, des diagrammes UML et des maquettes basse fidélité. Les deux premiers sprints font ici l'objet d'une illustration concrète. Ce document est une *version de conception* ; il ne constitue pas un bilan d'implémentation ou de tests réalisés.

**Mots-clés :** marketplace, animaux, Tunisie, UML, Spring Boot, Angular, PostgreSQL, Scrum, PFE.

# Abstract

This report presents the initial analysis and incremental design of **Souk El Hayawan**, a Tunisian web marketplace for legally permitted livestock and companion-animal listings. The selected scope includes sales, free adoption, donations, and a separate moderated area for lost, allegedly stolen, and found animals. The proposed technical foundation uses Angular, Spring Boot, PostgreSQL, JPA, and Flyway in a modular monolith. The report structures product requirements into a backlog and planned sprints, supported by UML specifications and low-fidelity wireframes. The document describes *planned work*, not a completed software implementation.

**Keywords :** animal marketplace, UML, Scrum, requirements, Spring Boot, Angular.

# Note méthodologique

**Conventions de lecture**

- VALIDÉ : décision fonctionnelle prise avec le porteur du projet.
- À CONFIRMER : option étudiée, non incluse dans le périmètre final sans validation.
- PLANIFIÉ : livrable d'un futur sprint, non réalisé à ce stade.
- Les schémas UML et maquettes de ce rapport sont des **propositions de conception**, et non des copies d'interfaces ou de code fonctionnels.

Ce rapport prend pour base les documents de cadrage et de backlog du projet. Les informations sur les plateformes concurrentes sont issues d'une première revue de leurs pages publiques ; elles ne constituent ni une expertise de sécurité, ni une enquête de terrain exhaustive. La présentation s'inspire des usages académiques du PFE, sans reprendre le contenu du rapport Farm Ghar.

# Table des matières

# Table des figures

# Liste des tableaux

CHAPITRE 1

# Introduction générale

## 1.1 Contexte

Le développement des outils numériques permet aux vendeurs et aux personnes intéressées par l'acquisition d'animaux de publier et de consulter des annonces à distance. En Tunisie, les besoins recouvrent à la fois les animaux d'élevage (ovins, caprins, bovins, équins, volailles) et les animaux de compagnie, dans le respect des règles de protection animale et des conditions légales de cession.

La variété des espèces soulève des questions de conception : tous les animaux ne disposent pas des mêmes caractéristiques pertinentes et toutes les annonces n'ont pas le même objectif. Une vente comporte un prix ; une adoption gratuite ou un don ne doivent pas dissimuler de contrepartie financière. Les alertes concernant un animal disparu répondent encore à un autre besoin et ne doivent pas être assimilées à des transactions commerciales.

## 1.2 Problématique

> **Comment concevoir une plateforme web tunisienne qui unifie les annonces de vente, d'adoption gratuite et de don pour plusieurs catégories d'animaux, propose une recherche multicritère fiable et offre un espace sécurisé pour les animaux perdus, présumés volés ou trouvés ?**

## 1.3 Objectifs

- Définir un **référentiel cohérent** d'espèces, de races et de localisations.
- Concevoir des **annonces typées** avec règles de saisie adaptées au contexte métier.
- Permettre une **recherche combinée** par type d'offre, caractéristiques et localisation.
- Séparer **les annonces commerciales** des **alertes d'entraide**.
- Assurer confidentialité, **contrôle d'accès** et modération des informations sensibles.
- Réaliser progressivement le projet afin de maîtriser concrètement Spring Boot, Angular, les bases de données et les diagrammes UML.

## 1.4 Démarche de travail

Le projet suit une approche incrémentale. Chaque sprint associe **besoin métier**, **conception**, **développement personnel**, **tests** et **recette**. Le développeur écrit lui-même son code ; l'accompagnement consiste à expliquer les choix, analyser les erreurs et vérifier les critères d'acceptation. Le démarrage reste volontairement simple : le monolithe modulaire précède les technologies avancées et l'éventuelle extraction de microservices.

## 1.5 Organisation du rapport

Le chapitre 2 présente l'existant ; le chapitre 3 décrit la solution proposée et les règles métier ; le chapitre 4 recense le backlog ; le chapitre 5 organise les sprints ; les chapitres 6 et 7 détaillent des exemples de dossiers de conception avec diagrammes UML et maquettes ; les perspectives de réalisation sont ensuite précisées.

CHAPITRE 2

# Étude de l'existant

## 2.1 Objectif et méthode

L'étude de l'existant vise à distinguer les fonctionnalités déjà présentes sur le marché tunisien des éléments différenciants envisagés pour Souk El Hayawan. Elle repose sur la consultation des pages publiques des plateformes recensées dans le document de cadrage le **19 octobre 2026**. Les constatations sont limitées aux fonctions présentées ou observables ; elles ne présument pas de l'architecture interne des sites.

## 2.2 Plateformes étudiées

### 2.2.1 Tayara — annonces généralistes

Tayara réunit des annonces de nombreux domaines, dont une rubrique consacrée aux animaux. Elle offre un parcours généraliste de publication et de recherche d'offres. Cette polyvalence est un atout, mais elle ne correspond pas à une spécialisation exclusive sur les fiches propres aux animaux d'élevage et de compagnie.

Source : https://www.tayara.tn/

### 2.2.2 Sawt El Falleh — marché agricole

Sawt El Falleh s'adresse au secteur agricole tunisien. Les fonctionnalités décrites dans le cadrage comprennent des annonces de vente et d'achat, des catégories, des filtres régionaux et de prix, ainsi que le contact direct. Il s'agit d'un point de comparaison important : **rechercher un animal par prix et par région n'est pas en soi une innovation**.

Source : https://www.sawtelfalleh.tn/

### 2.2.3 TunisieVet — annonces et orientation vétérinaire

TunisieVet propose un espace d'annonces d'animaux et des contenus associés aux services vétérinaires. Son formulaire de vente prévoit des informations comme l'espèce, la race, l'âge, le poids, les photos, le prix et la localisation. La structuration des données animales existe donc également dans cette offre.

Source : https://tunisievet.com/fr/vente-animaux

### 2.2.4 Animalia — animaux de compagnie

Animalia est orientée vers les animaux de compagnie et l'adoption responsable. Ses pages de présentation mettent en avant les annonces et des engagements liés à la vérification. Cette orientation est directement pertinente pour la partie « compagnie » de notre futur produit.

Source : https://www.animalia.tn/

## 2.3 Comparaison qualitative

**Table 2.1.** Positionnement des plateformes étudiées

| Plateforme | Orientation relevée | Apport pour notre conception |
|---|---|---|
| Tayara | Petites annonces généralistes | Comprendre un parcours de consultation multi-catégories. |
| Sawt El Falleh | Échanges agricoles ; filtres catégorie, région et prix | Consolider des fiches animales plus adaptées aux espèces. |
| TunisieVet | Annonces animales et informations vétérinaires | S'inspirer de données structurées par animal. |
| Animalia | Annonces et adoption d'animaux de compagnie | Étudier un parcours d'adoption et de modération. |

> **Limite de l'étude :** l'absence d'une fonctionnalité sur une page visitée ne prouve pas son absence de la plateforme. Il serait incorrect d'affirmer qu'aucun concurrent ne propose la recherche par prix, les races ou les filtres géographiques.

## 2.4 Opportunité identifiée

Le positionnement envisagé repose sur la **cohérence d'un parcours multi-espèces** plutôt que sur une fonctionnalité isolée : annonces adaptées à la nature de l'offre, données comparables, filtres combinables, prise en charge des régions tunisiennes, et espace spécifique d'entraide. Une étude utilisateur complémentaire reste nécessaire pour confirmer que cet ensemble apporte une valeur réellement perçue.

## 2.5 Conclusion de l'étude

L'étude montre un marché déjà occupé par des services généralistes ou spécialisés. Souk El Hayawan devra donc être évalué sur la pertinence de ses parcours, la qualité de ses données, la simplicité de l'expérience et sa modération, plutôt que sur une prétention de première plateforme du secteur.

CHAPITRE 3

# Analyse des besoins et solution proposée

## 3.1 Vue d'ensemble

**Souk El Hayawan** vise un système de mise en relation en Tunisie, accessible sur téléphone et ordinateur, avec interface prévue en français, arabe tunisien (support RTL) et anglais. Les catégories couvrent dès la première version les animaux d'élevage *et* les animaux de compagnie dont la publication ou la cession est autorisée.

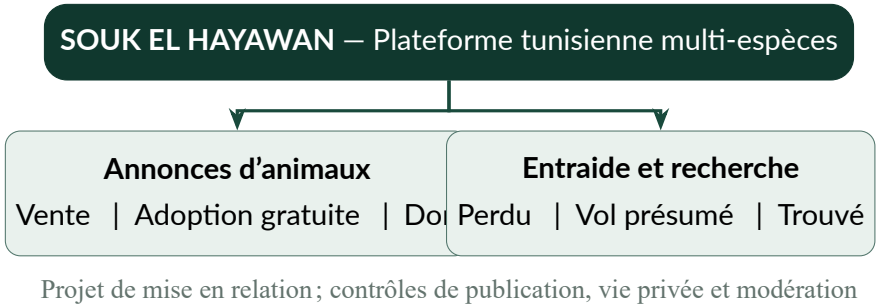



**Figure 3.1.** Deux espaces métier clairement séparés — proposition de conception

## 3.2 Acteurs identifiés

**Table 3.1.** Acteurs principaux et responsabilités

| Acteur | Fonctions attendues |
|---|---|
| Visiteur | Parcourir les annonces publiques, rechercher par type d'offre, animal, région et prix, consulter les avis d'entraide. |
| Membre | Gérer son profil, enregistrer des favoris, échanger dans un canal privé, publier selon ses droits. |
| Auteur d'annonce | Créer et gérer une annonce de vente, adoption gratuite ou don, joindre des photos, suivre son statut. |
| Déclarant / trouveur | Publier une alerte PERDU, VOL PRÉSUMÉ ou TROUVÉ, dialoguer et clôturer après résolution. |
| Modérateur | Vérifier les annonces et avis sensibles, traiter les signalements et documenter les décisions. |
| Administrateur | Gérer les référentiels, les habilitations et la configuration métier. |

## 3.3 Périmètre fonctionnel validé

### 3.3.1 Annonces d'animaux

Trois natures d'offres sont VALIDÉES :

- **Vente** : l'annonce porte un prix strictement positif, exprimé en TND.
- **Adoption gratuite** : recherche d'un nouveau foyer, sans paiement ni prix caché.
- **Don** : cession gratuite sans contrepartie financière ; le sens juridique et fonctionnel exact vis-à-vis de l'adoption doit figurer dans les conditions d'utilisation.

Une annonce possède un auteur, un type, une espèce, des caractéristiques pertinentes, une localisation approximative et un état. Le prix ne s'applique qu'à une vente. Les espèces protégées ou dont la commercialisation est interdite sont exclues.

### 3.3.2 Entraide pour animaux disparus et trouvés

- **J'ai perdu un animal** : photo, description et zone approximative de disparition.
- **Vol présumé** : même parcours de recherche, sans désigner publiquement un suspect.
- **J'ai trouvé un animal** : avis de découverte, permettant au propriétaire potentiel de se manifester de manière confidentielle.

Ces avis sont **distincts des annonces de cession** : aucun prix, une modération avant diffusion, pas d'adresse précise publique, et une procédure prudente pour vérifier la propriété avant restitution.

### 3.3.3 Reproduction / saillie — besoin à confirmer

Le projet étudie une mise en relation entre propriétaires recherchant un partenaire pour la reproduction de leur animal. Ce parcours, parfois désigné « tchatel », est À CONFIRMER avant intégration au MVP. Le sens du terme, les conditions sanitaires, l'âge, les règles par espèce et l'éventuelle rémunération ne sont pas arrêtés. Il ne doit pas être traité comme une vente de l'animal.

## 3.4 Exigences fonctionnelles

**Table 3.2.** Exigences fonctionnelles majeures

| Code | Domaine | Besoin |
|---|---|---|
| BF-01 | Catalogue | Consulter les espèces et races actives. |

| Code | Domaine | Besoin |
|---|---|---|
| BF-02 | Annonces | Rechercher et ouvrir une annonce de vente, adoption ou don. |
| BF-03 | Recherche | Combiner espèce, race, type, gouvernorat, délégation et prix si applicable. |
| BF-04 | Comptes | Créer un compte, se connecter et gérer son profil. |
| BF-05 | Publication | Ajouter, modifier, publier, suspendre ou clôturer ses propres annonces. |
| BF-06 | Communication | Contacter un autre membre dans un canal privé autorisé. |
| BF-07 | Entraide | Signaler un animal perdu ou présumé volé. |
| BF-08 | Entraide | Déclarer un animal trouvé et permettre une restitution prudente. |
| BF-09 | Modération | Examiner les signalements et masquer les publications problématiques. |
| BF-10 | Administration | Gérer le catalogue, les rôles et les données de référence. |
| BF-11 | Langues | Présenter une interface FR / ar-TN / EN, avec RTL pour l'arabe. |
| BF-12 | Évolution | Étudier une recherche de partenaire pour reproduction sous réserve de validation. |

## 3.5 Règles métier essentielles

**RG-1.** Une race est associée à une catégorie d'animaux ; une délégation appartient à un gouvernorat.

**RG-2.** Une annonce de **vente** exige un prix positif, stocké en valeur décimale exacte (`BigDecimal / NUMERIC`).

**RG-3.** Une annonce d'**adoption gratuite** ou de **don** ne peut contenir ni prix de vente ni frais dissimulés.

**RG-4.** Une annonce **brouillon** ou **archivée** n'apparaît pas dans la recherche publique.

**RG-5.** Seul l'auteur autorisé peut modifier son annonce ; les fonctions de modération et d'administration ont des permissions distinctes.

**RG-6.** Les filtres sont combinables ; lorsque les deux bornes existent, le prix minimal ne peut excéder le maximal.

**RG-7.** Les avis PERDU / VOL PRÉSUMÉ / TROUVÉ sont modérés avant diffusion et ne divulguent ni documents sensibles ni localisation exacte.

**RG-8.** Un soupçon de vol n'est jamais assimilé à une preuve de culpabilité ; aucun suspect

n’est désigné automatiquement.

**RG-9.** Une restitution est précédée d’une vérification raisonnable et confidentielle de l’appartenance.

**RG-10.** L’application refuse les catégories d’animaux dont les échanges sont légalement interdits.

## 3.6 Exigences non fonctionnelles

- **Utilisabilité** : responsive mobile et desktop, formulaires compréhensibles, navigation cohérente.
- **Multilinguisme** : français, arabe tunisien et anglais ; direction RTL lorsque nécessaire.
- **Sécurité** : contrôle des autorisations côté backend, protection des données privées, stockage sûr des secrets.
- **Qualité** : API documentée, validation, tests, migrations de base de données versionnées.
- **Évolutivité** : monolithe modulaire et découpage métier clair, sans microservices prématurés.

## 3.7 Architecture cible pour le démarrage

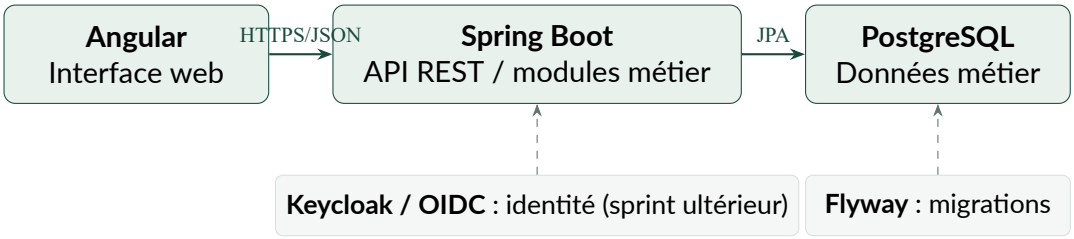



**Figure 3.2.** Architecture logique envisagée : monolithe modulaire et API REST

L’interface sera réalisée avec **Angular** ; l’API et les règles métier avec **Spring Boot** ; les données persistées dans **PostgreSQL**, sous contrôle des migrations **Flyway** et de l’accès JPA/Hibernate. L’authentification pourra s’appuyer sur **Keycloak/OIDC**. Ces technologies décrivent la trajectoire de développement, pas un état déjà livré.

## 3.8 Cas d’utilisation global — aperçu

> Les diagrammes sont des **modèles à valider**. Les relations UML détaillées, les scénarios d’exception et la traçabilité vers les exigences seront enrichis sprint par sprint, avec preuve de développement et tests à partir de la phase de réalisation.

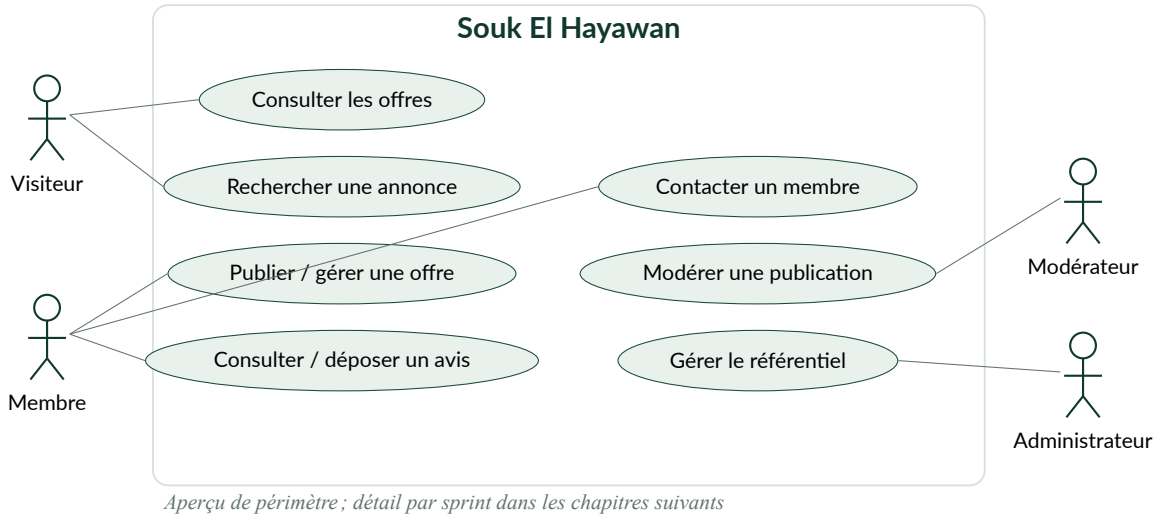



**Figure 3.3.** Cas d'utilisation métier principaux de Souk El Hayawan

CHAPITRE 4

# Backlog produit global

## 4.1 Organisation et priorités

Le backlog constitue la source de référence des fonctionnalités. Il est structuré en **epics** et **user stories** ; chaque sprint sélectionne un ensemble cohérent de stories. Les identifiants de la version Drive existante sont conservés pour faciliter la traçabilité.

> **Priorités :** M indispensable au MVP ; S souhaitable ou phase ultérieure ; C amélioration de long terme. Une story marquée À confirmer ne devient pas un engagement de sprint par défaut.

**Table 4.1.** Backlog global — version de conception, identifiants alignés sur le Drive

| ID | Prio. | User story synthétisée |
|---|---|---|
| **E01 — Fondations et interface** | | |
| US-001 | M | Consulter une page d'accueil responsive sans fonctionnalités simulées. |
| US-002 | M | Démarrer Spring Boot, Angular, Git et environnement local documenté. |
| US-003 | M | Fournir une API de santé et une structure applicative claire. |
| US-004 | S | Changer de langue FR / ar-TN / EN et gérer le sens RTL. |
| **E02 — Catégories et races** | | |
| US-010 | M | Consulter les catégories d'animaux d'élevage et de compagnie et leurs races. |
| US-011 | M | Garantir l'intégrité des relations espèce–race dans PostgreSQL. |
| US-012 | M | Administrer l'ajout, l'édition et la désactivation des catégories/races. |
| US-013 | S | Afficher le catalogue avec traductions dans trois langues. |
| **E03 — Localisation** | | |
| US-020 | M | Choisir un gouvernorat issu d'un référentiel vérifié. |
| US-021 | M | Afficher les délégations du gouvernorat sélectionné. |
| US-022 | S | Importer ou corriger un référentiel géographique traçable. |
| US-023 | S | Explorer les annonces sur une carte ou par rayon. |

| ID | Prio. | User story synthétisée |
|---|---|---|
| **E04 — Comptes et habilitations** | | |
| US-030 | M | Créer un compte et se connecter / déconnecter. |
| US-031 | M | Gérer les informations personnelles autorisées. |
| US-032 | M | Appliquer les droits membre, modérateur et administrateur. |
| US-033 | M | Permettre à l'auteur de gérer uniquement ses annonces. |
| **E05 — Annonces et recherche** | | |
| US-040 | M | Créer, publier, modifier, suspendre ou clôturer une annonce de vente, adoption gratuite ou don. |
| US-041 | M | Joindre photos et caractéristiques adaptées à l'espèce et au type d'offre. |
| US-042 | M | N'afficher publiquement que les annonces publiées. |
| US-043 | M | Rechercher par offre, espèce, race, prix applicable et région. |
| US-044 | M | Trier, paginer et consulter une fiche détaillée. |
| US-045 | S | Partager une URL de recherche persistante. |
| US-046 | S | Suivre les vues et le statut de ses annonces. |
| US-047 | M | Publier adoption gratuite ou don, sans prix ni contrepartie cachée. |
| US-048 | À confirmer | Rechercher ou proposer une mise en relation pour reproduction/saillie. |
| US-049 | M | Valider prix, types d'offres et catégories autorisées. |
| **E06 — Échanges et notifications** | | |
| US-050 | M | Ajouter / retirer des annonces des favoris. |
| US-051 | M | Contacter l'auteur d'une annonce dans une conversation privée. |
| US-052 | M | Répondre aux conversations autorisées. |
| US-053 | S | Enregistrer une recherche et recevoir des alertes sans doublon. |
| **E07 — IA, modération et entraide** | | |
| US-060 | S | Proposer un classement explicable des annonces pertinentes. |
| US-061 | S | Transformer une recherche libre multilingue en filtres confirmables. |
| US-062 | S | Assister la rédaction d'annonce sous validation humaine. |
| US-063 | M avant ouverture | Traiter les signalements et contrôler les contenus. |
| US-064 | S | Comparer des prix à partir de données disponibles, de manière indicative. |
| US-065 | M | Déposer un avis PERDU ou VOL PRÉSUMÉ. |
| US-066 | M | Modérer les avis PERDU / VOL PRÉSUMÉ / TROUVÉ. |
| US-067 | M | Rechercher les avis d'entraide et transmettre une observation en privé. |

| ID | Prio. | User story synthétisée |
|---|---|---|
| US-068 | M | Clôturer un avis après retrouvailles ou restitution vérifiée. |
| US-069 | S | Suggérer plus tard des rapprochements entre avis, sous revue humaine. |
| US-076 | M | Déclarer « J'ai trouvé un animal » avec photo et lieu approximatif. |
| US-077 | M | Échanger en privé et vérifier l'appartenance avant restitution. |
| **E08 — Technologies avancées et qualité** | | |
| US-070 | S | Importer de gros référentiels via Spring Batch. |
| US-071 | C | Proposer une lecture GraphQL ciblée en complément de REST. |
| US-072 | C | Intégrer une API externe de manière non bloquante si nécessaire. |
| US-073 | M | Automatiser les tests, builds et livraisons. |
| US-074 | S | Exposer des métriques, logs et états de santé utiles. |
| US-075 | C | Extraire un module stable en microservice sans régression. |

## 4.2 Règles de suivi du backlog

- Chaque user story conserve un **identifiant stable**.
- La priorité est révisable selon la faisabilité et la valeur métier.
- Les stories de **reproduction** restent provisoires tant que le périmètre n'est pas confirmé.
- Les fonctionnalités de **modération minimale** peuvent être avancées pour sécuriser une mise en ligne.
- Une story n'est « terminée » qu'après code fonctionnel, cas d'erreur, tests et démonstration.

CHAPITRE 5

# Organisation du développement en sprints

## 5.1 Méthode de découpage

Les sprints suivent un principe de construction cumulative : le catalogue et la localisation précèdent les annonces ; les comptes et autorisations précèdent l'édition des contenus ; les annonces réelles précèdent les favoris, la recherche avancée et les notifications. La numérotation suivante constitue une **planification indicative**, pas une exécution passée.




Chaque étape réutilise les acquis des sprints précédents

**Figure 5.1.** Progression planifiée du monolithe vers des fonctionnalités avancées

## 5.2 Roadmap synthétique

**Table 5.1.** Planification S00 à S16 et sprint 08 bis

| Sprint | Objectif | Livrable / compétence dominante | Stories |
|---|---|---|---|
| **S00** | **Démarrage du projet** | Git, Spring Boot, Angular, page d'accueil réelle, API de santé. | 001–003 |
| **S01** | **Catégories / races** | PostgreSQL, Flyway, JPA, API de lecture et écran catalogue. | 010, 011, 013 |
| **S02** | **Localisation** | Gouvernorats, délégations, sélection dépendante et données sourcées. | 020, 021 |
| **S03** | **Sécurité** | Inscription, connexion, Keycloak/OIDC, rôles et permissions. | 030–032 |
| **S04** | **Back-office** | Administration des catégories/races et audit minimal. | 012, 022 |
| **S05** | **Publication** | Vente, adoption gratuite, don, photos, états, contrôle de propriété. | 040–042, 047, 049 |
| **S06** | **Recherche** | Filtres combinés, pagination et recherche réelle sur annonces publiées. | 043–045 |

| Sprint | Objectif | Livrable / compétence dominante | Stories |
|---|---|---|---|
| **S07** | **Échanges** | Favoris et conversations privées avec autorisations. | 050–052 |
| **S08** | **Alertes de recherche** | Requêtes enregistrées, Scheduler, notification sans doublon. | 053 |
| **S08 bis** | **Entraide animale** | Avis PERDU / VOL PRÉSUMÉ / TROUVÉ, modération, restitution. | 065–068, 076, 077 |
| **S09** | **Carte et proximité** | Recherche géographique ; PostGIS si nécessaire. | 023 |
| **S10** | **Matching métier** | Classement explicable et éventuelles demandes d'achat. | 060 |
| **S11** | **Assistant IA** | Recherche textuelle FR / ar-TN / Arabizi / EN et validation des filtres. | 061, 062 |
| **S12** | **Confiance et reporting** | Modération renforcée et comparaisons de prix indicatives. | 063, 064 |
| **S13** | **Rapprochement prudent** | Suggestions entre avis disparus et trouvés, avec revue humaine. | 069 |
| **S14** | **Technologies ciblées** | Spring Batch, GraphQL, client asynchrone sur cas justifiés. | 070–072 |
| **S15** | **Industrialisation** | Tests intégrés, Docker, CI/CD et observabilité. | 073, 074 |
| **S16** | **Microservices** | Extraction d'un module stable, contrats et résilience. | 075 |

## 5.3 Dépendances métier et techniques

- Le référentiel **espèce / race** précède les annonces qui en dépendent.
- Le référentiel **gouvernorat / délégation** précède les recherches régionales.
- Les **autorisations** précèdent la gestion privée des annonces et le back-office.
- Une **recherche réelle** suppose des annonces persistées et un état de publication contrôlé.
- La **séparation en microservices** ne sera étudiée qu'après stabilisation de tests et des frontières métier.

## 5.4 Livrables de conception exigés pour chaque sprint

## 5.5 Définition de « terminé » (DoD)

Une fonctionnalité est considérée comme achevée seulement si : **(1)** le besoin est expliqué, **(2)** les interfaces et modèles sont cohérents, **(3)** le code a été produit et revu, **(4)** les scénarios

**Table 5.2.** Pack de conception et de recette à produire sprint par sprint

| N° | Livrable | Contenu attendu |
|---|---|---|
| 01 | Fiche de sprint | Objectif, périmètre, stories, dépendances et critères de recette. |
| 02 | Diagramme de cas d'utilisation | Image UML regroupant les acteurs et cas d'utilisation du sprint. |
| 03 | Fiches de cas d'utilisation | Tableau par cas : pré/postconditions, scénario nominal, alternatives, erreurs. |
| 04 | Diagrammes de séquence | Un diagramme par cas d'utilisation détaillé significatif. |
| 05 | Diagramme de classes | Classes concernées, attributs, liens et multiplicités. |
| 06 | Diagrammes d'activité | Parcours métier et décisions, surtout pour les cas complexes. |
| 07 | Maquettes basse fidélité | Écrans façon Balsamiq, sans se confondre avec l'UI finale. |
| 08 | Règles / tests / démonstration | Traçabilité story–test, validations, captures et bilan de sprint. |

nominal, erreur et sécurité sont testés, **(5)** la démonstration fonctionne avec des données réelles lorsqu'elles sont nécessaires, et **(6)** la documentation et le commit sont disponibles.

CHAPITRE 6

# Sprint S00 — Initialisation du projet

## 6.1 Fiche de sprint

**Table 6.1.** Cadrage du sprint S00

| Champ | Description |
|---|---|
| Objectif | Créer un nouveau dépôt Git et un socle technique propre avec une page d'accueil responsive et une API de santé. |
| Périmètre | Spring Boot, Angular, Git, configuration locale, README, architecture simple ; pas de base de données ni d'authentification métier. |
| Stories | `US-001, US-002, US-003`. |
| Résultat attendu | Backend et frontend qui démarrent ; contenu descriptif sans fausses annonces. |
| Préconditions | Environnement Java/Node/Git vérifié, compatibilité Angular et Node confirmée. |
| Recette | GET santé en HTTP 200 ; frontend compilé ; page responsive ; README reproductible ; dépôt sans secrets. |
| Statut | PLANIFIÉ — démarrage après validation du cadrage. |

## 6.2 Diagramme de cas d'utilisation du sprint

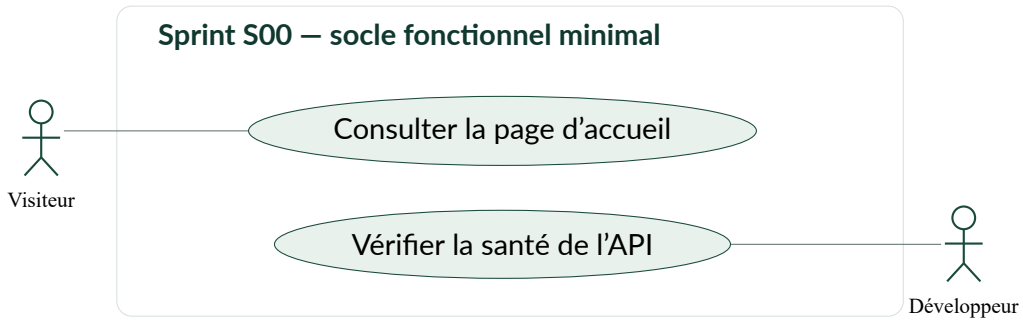



**Figure 6.1.** S00 — diagramme de cas d'utilisation

L'initialisation technique n'est pas un parcours commercial ; les deux cas du sprint concernent la **présentation du service** et la **vérification de disponibilité**. Les cas de catalogue persistant commencent au sprint S01.

## 6.3 UC-S00-01 — Consulter la page d'accueil

**Table 6.2.** Description du cas UC-S00-01

| Rubrique | Description |
|---|---|
| **Code** | UC-S00-01 |
| **Acteur principal** | Visiteur |
| **Objectif** | Lire la présentation de Souk El Hayawan et comprendre les services envisagés. |
| **Précondition** | Le frontend est lancé et son URL est accessible. |
| **Déclencheur** | Le visiteur ouvre la page d'accueil. |
| **Scénario nominal** | 1. Le navigateur charge l'application.<br>2. Angular initialise la page.<br>3. Le visiteur consulte l'identité, le périmètre et les catégories de services. |
| **Alternative** | En cas de chargement impossible, le navigateur présente un état d'erreur. |
| **Postcondition** | La page descriptive est affichée sans résultat de recherche fictif. |
| **Critère de recette** | Affichage desktop/mobile sans débordement ; pas de bouton prétendant appeler une fonctionnalité inexistante. |

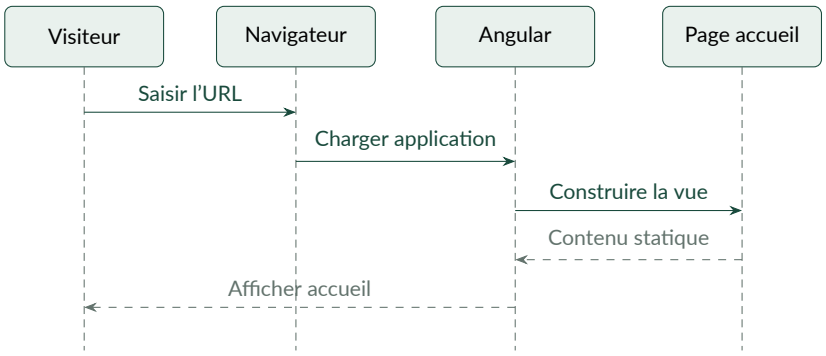



**Figure 6.2.** UC-S00-01 — diagramme de séquence de consultation

## 6.4 UC-S00-02 — Vérifier la disponibilité de l'API

**Table 6.3.** Description du cas UC-S00-02

| Rubrique | Description |
|---|---|
| **Code** | UC-S00-02 |
| **Acteur principal** | Développeur |
| **Objectif** | Vérifier que Spring Boot est accessible via une route de santé. |
| **Précondition** | Le serveur backend est démarré et son port est connu. |
| **Déclencheur** | Envoi d'une requête GET vers l'endpoint de santé. |
| **Scénario nominal** | 1. Le développeur envoie la requête.<br>2. Le contrôleur construit un statut.<br>3. Le client reçoit le code HTTP 200 et un JSON exploitable. |
| **Alternative** | Backend indisponible : erreur de connexion ou réponse non conforme. |
| **Postcondition** | L'état de disponibilité est objectivement vérifié. |
| **Critère de recette** | Code HTTP 200, contrat de réponse documenté et commande de test reproductible. |

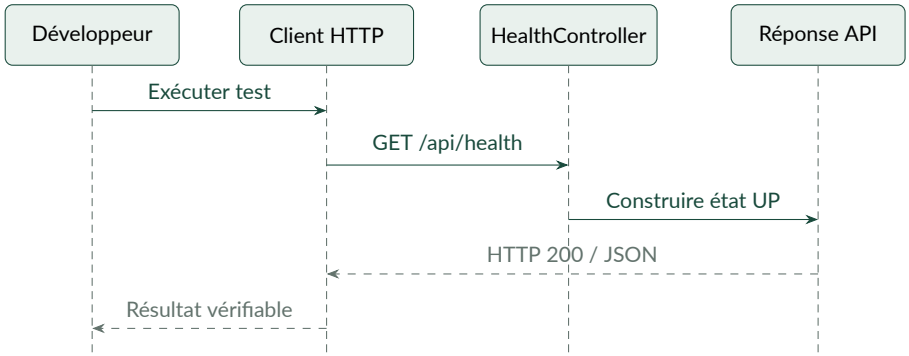
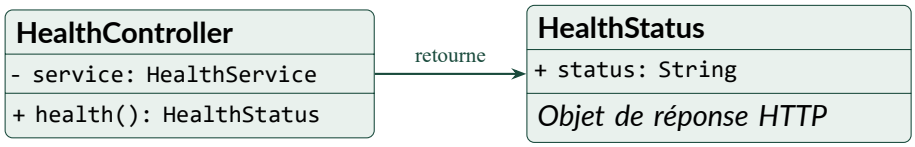



**Figure 6.3.** UC-S00-02 — diagramme de séquence de vérification API

## 6.5 Diagramme de classes de conception




Modèle pédagogique de conception : aucun objet métier ni couche JPA au Sprint 0

**Figure 6.4.** S00 — premier diagramme de classes technique, sans données métier

## 6.6 Diagramme d'activité

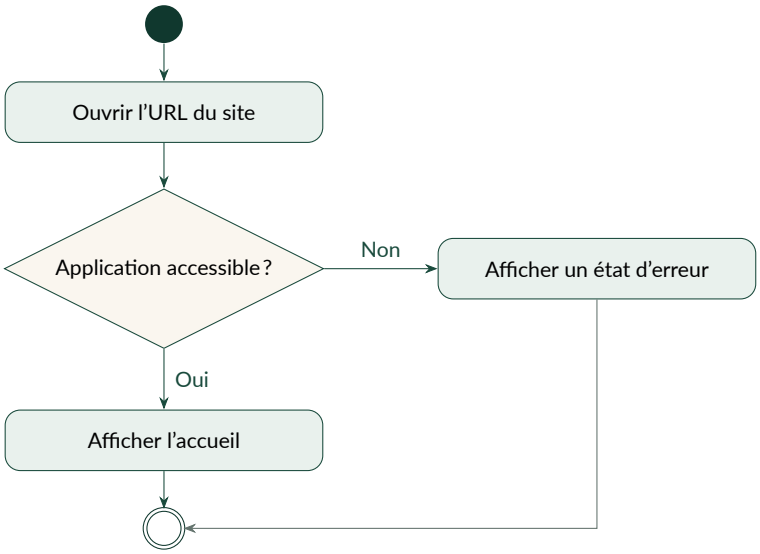



**Figure 6.5.** S00 — activité de consultation de l'accueil

## 6.7 Maquette de l'accueil — style Balsamiq




**Figure 6.6.** S00 — maquette basse fidélité de la landing page (illustration)

> **Décision de conception :** S00 ne présente pas de faux comptes ou de fausses annonces. La maquette représente une page *informative*. Les écrans de catalogue reliés à la base réelle interviennent en S01.

## 6.8 Travaux du développeur et validation

1. Rédiger trois parcours utilisateur avec ses propres mots.
2. Créer le dépôt Git, organiser `frontend/` et `backend/`, configurer `.gitignore`.
3. Initialiser Spring Boot et créer l'endpoint de santé.
4. Initialiser Angular, SCSS et les composants de base.
5. Réaliser l'accueil, vérifier le responsive et les liens actifs.
6. Produire un README et une démonstration de fonctionnement.

> **À compléter après réalisation :** résultats des tests, captures de l'application développée, détails d'implémentation et bilan personnel du sprint S00. Aucune fonctionnalité non développée n'est présentée comme validée.

CHAPITRE 7

# Sprint S01 — Référentiel des animaux

## 7.1 Fiche de sprint

**Table 7.1.** Cadrage du sprint S01

| Champ | Description |
|---|---|
| Objectif | Fournir un catalogue réel d'espèces et de races, stocké en base, consultable par API puis par Angular. |
| Stories | `US-010`, `US-011`, `US-013`. |
| Préconditions | Socle S00 validé, PostgreSQL disponible, modèle relationnel accepté. |
| Livrables | Migrations Flyway, entités JPA, API GET de catégories/races, écran de consultation. |
| Vérification | Les valeurs affichées proviennent de PostgreSQL ; contraintes d'intégrité et traductions vérifiées. |
| Statut | PLANIFIÉ — diagrammes de conception préliminaires. |

## 7.2 Use cases du sprint

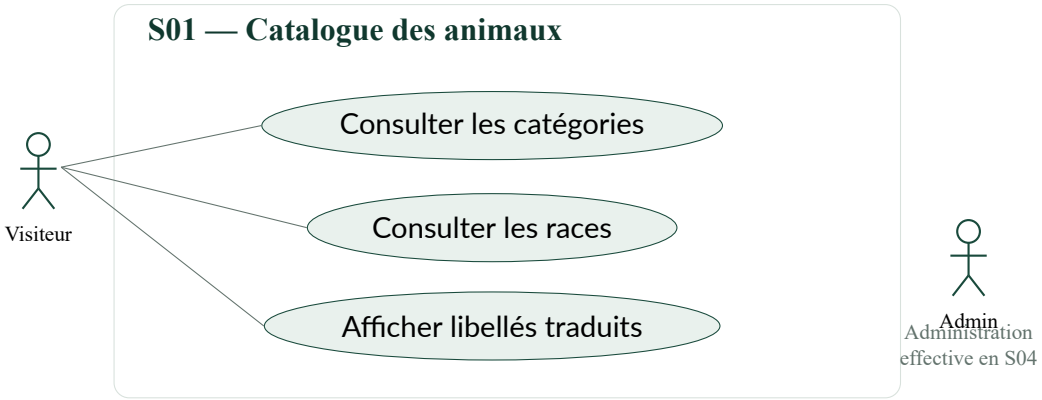



**Figure 7.1.** S01 — diagramme de cas d'utilisation du référentiel

## 7.3 UC-S01-01 — Consulter les races d'une catégorie

**Table 7.2.** Description du cas UC-S01-01

| Rubrique | Description |
|---|---|
| **Code** | UC-S01-01 |
| **Acteur principal** | Visiteur |
| **Objectif** | Consulter les races actives d'une catégorie d'animaux. |
| **Préconditions** | Le catalogue persistant contient des catégories valides ; le service de consultation est disponible. |
| **Déclencheur** | Choix d'une catégorie dans l'interface. |
| **Scénario nominal** | 1. Charger les catégories actives.<br>2. Sélectionner une catégorie.<br>3. Angular appelle l'API des races.<br>4. Le backend vérifie la catégorie et retourne les races actives.<br>5. L'interface affiche la liste. |
| **Alternatives** | Catégorie inconnue : message adapté ; aucune race : état vide explicite ; API indisponible : gestion d'erreur sans fausses données. |
| **Règles métier** | Une race appartient à une catégorie ; seules les catégories et races actives sont visibles. |
| **Postconditions** | Le visiteur voit les races réellement présentes dans la base. |
| **Critères** | API et interface synchronisées ; changement de catégorie actualise les races ; tests des états vide/invalide. |

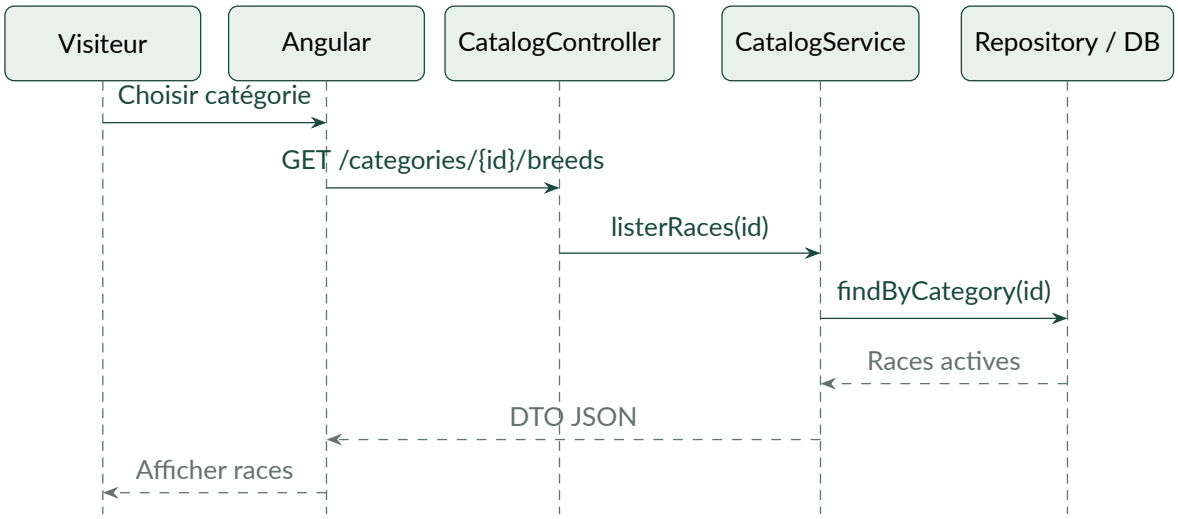



**Figure 7.2.** UC-S01-01 — diagramme de séquence Angular / REST / JPA

## 7.4 Modèle de classes préliminaire

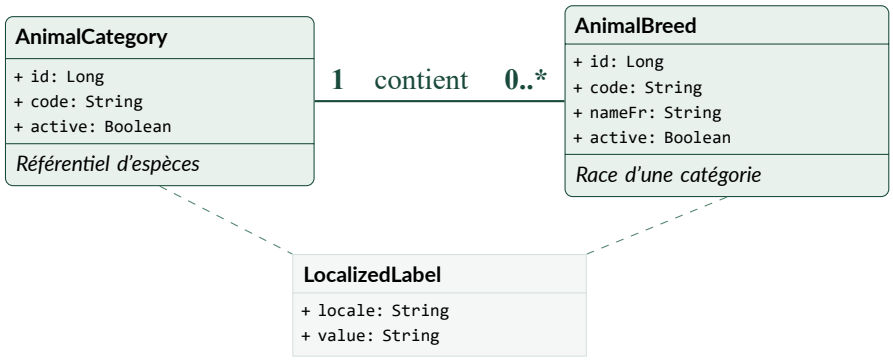



**Figure 7.3.** S01 — diagramme de classes conceptuel des catégories et races

Le modèle propose une relation **une catégorie vers plusieurs races**. L'entité de libellé localisé est une *option de conception*, à valider selon la stratégie choisie pour les trois langues. Aucun code de race ni aucune liste géographique ne doit être inventé pour remplir la base.

## 7.5 Diagramme d'activité

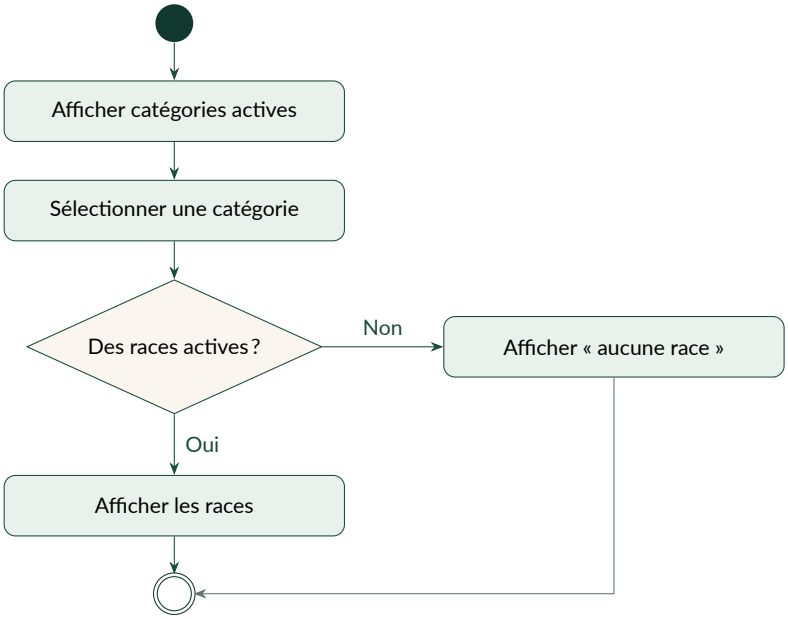



**Figure 7.4.** S01 — consultation d'une catégorie et de ses races

## 7.6 Maquette de catalogue — style Balsamiq

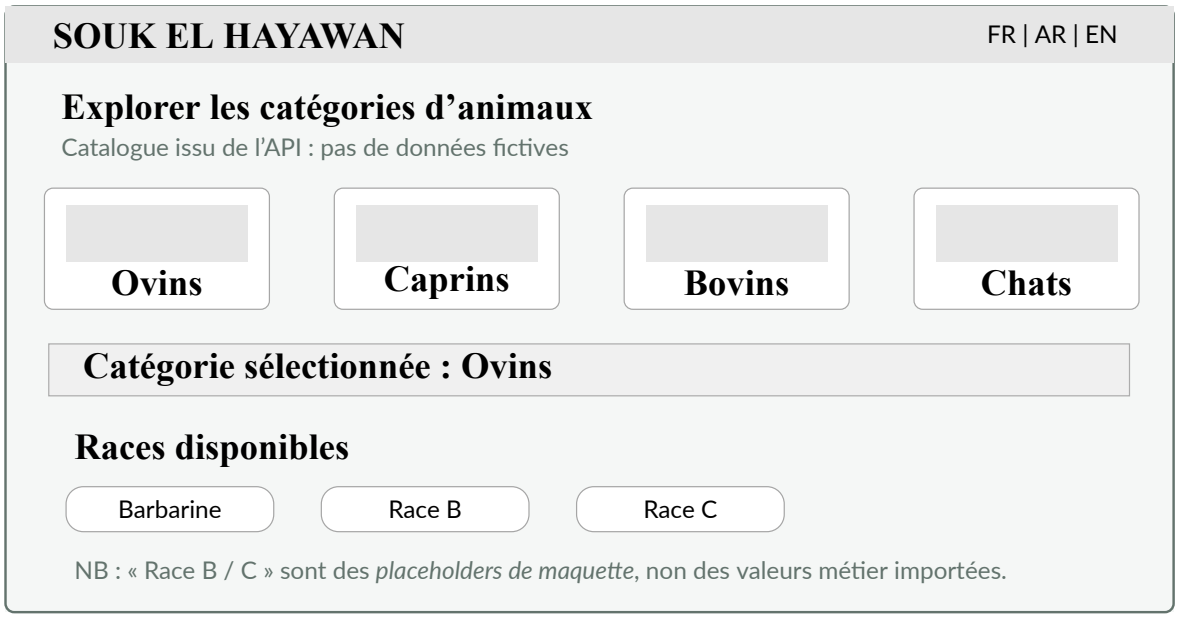



**Figure 7.5.** S01 — wireframe de consultation des espèces et races

## 7.7 Tests attendus

- Catégories actives listées ; catégories inactives absentes du résultat public.
- Races d'une catégorie obtenues par une requête réellement exécutée.
- Catégorie inconnue et catégorie vide traitées sans affichage trompeur.
- Intégrité relationnelle vérifiée par contrainte de clé étrangère.
- Données de référence sourcées et migrations reproductibles.
- Interface Angular raccordée à la vraie API ; tests manuels et automatisés.

> **À compléter après réalisation :** scripts Flyway rédigés par le développeur, tests JUnit, captures Angular et démonstration du sprint S01. Aucune fonctionnalité non développée n'est présentée comme validée.

CHAPITRE 8

# Perspectives et préparation de la réalisation

## 8.1 Un dossier de conception par sprint

Après validation du backlog, chaque sprint doit recevoir les mêmes rubriques que les exemples S00 et S01 : fiche de sprint, use case général, fiches détaillées, séquences par cas, classes, activités, maquettes basse fidélité, règles et tests. Les diagrammes ne seront pas copiés à l'identique : seuls les acteurs, objets et interactions utiles au sprint concerné apparaîtront.

> **Principe de traçabilité :** `US-040` → `S05` → `UC-S05-01` → diagramme de séquence → classes/écran → tests et preuves d'acceptation.

## 8.2 Points de cadrage restant ouverts

- Sens et périmètre métier de la **reproduction/saillie** (« tchatel »), et éventuelle rémunération.
- Politique de **contact** : messagerie interne, WhatsApp ou combinaison encadrée.
- Délimitation opérationnelle entre **adoption gratuite** et **don**.
- Processus de **modération** initiale, pièces justificatives recevables et conservation des informations sensibles.
- Mise en œuvre du **multilinguisme**, notamment expressions tunisiennes et orientation RTL.
- Source et qualité du **référentiel géographique** tunisien.
- Conditions de prise en charge des **espèces dont la détention ou la cession est réglementée**.

## 8.3 Trajectoire technique

**Étape 1 :** maîtriser le monolithe modulaire, les API REST, les migrations, l'authentification et les tests. **Étape 2 :** introduire uniquement les outils avancés justifiés par le besoin (Scheduler, Batch, géographie, matching). **Étape 3 :** étudier l'extraction de microservices après stabilisation des frontières métier et de la chaîne de tests.

## 8.4 Conclusion provisoire

Le projet dispose d'une vision, d'une étude comparative initiale, d'une solution proposée, d'un backlog structuré et d'une roadmap. Le présent dossier présente des **modèles de conception préliminaires**. Il sera complété de manière incrémentale à chaque sprint, en distinguant les choix validés des fonctionnalités réellement développées, testées et démontrées.

CHAPITRE A

# Modèle vierge de dossier de sprint

> **Mode d'emploi :** copier ce modèle dans un nouveau fichier `chapters/sprintXX.tex`, compléter les sections et produire les figures TikZ associées dans `figures/`. Les exemples S00 et S01 montrent la mise en page attendue.

## A.1 Fiche de sprint à remplir

**Table A.1.** Gabarit de fiche de sprint

| Champ | À renseigner |
|---|---|
| Sprint / version | Code, période et statut. |
| Objectif | Résultat utilisateur observable. |
| Stories incluses | Identifiants stables depuis le backlog. |
| Cas d'utilisation | UC-SXX-01, UC-SXX-02, etc. |
| Architecture | Classes, API, écrans et relations nécessaires. |
| Livrables | Code, maquettes, diagrammes, documentation. |
| Tests | Scénario nominal, erreurs, permissions, régressions. |
| Critère d'acceptation | Conditions démontrables, sans données fictives. |

## A.2 Fiche détaillée de cas d'utilisation — gabarit

## A.3 Livrables graphiques à joindre

1. **Use case du sprint** : acteurs et fonctionnalités de l'incrément.
2. **Séquence de chaque cas détaillé important** : messages et réponses des composants.
3. **Diagramme de classes** : attributs, associations, multiplicités.
4. **Diagramme d'activité** : étapes, conditions et parcours d'erreur.
5. **Maquettes** : page d'accueil, liste, détail, formulaires, back-office ou autres écrans pertinents.

**Table A.2.** Trame standard des cas d'utilisation

| Champ | Description à compléter |
|---|---|
| Code / intitulé | UC-SXX-YY : verbe et objet métier. |
| Acteur | Acteur principal et acteurs secondaires. |
| But | Valeur recherchée. |
| Préconditions | État requis avant exécution. |
| Déclencheur | Action initiatrice. |
| Scénario nominal | Séquence numérotée : actions acteur / réponses système. |
| Alternatives | Parcours valides de remplacement. |
| Exceptions | Erreurs, permissions et indisponibilités. |
| Postconditions | Effet sur l'état système. |
| Règles métier | Identifiants RG-XX et validations. |
| Critères de recette | Tests vérifiables, succès / échec. |

CHAPITRE B

# Sources et ressources de référence


1. Documents de cadrage et backlog de **Souk El Hayawan** conservés dans Google Drive (documents de travail).
2. Tayara : https://www.tayara.tn/.
3. Sawt El Falleh : https://www.sawtelfalleh.tn/.
4. TunisieVet : https://tunisievet.com/fr/vente-animaux.
5. Animalia : https://www.animalia.tn/.
6. *Farm Ghar*, rapport PFE fourni par l'utilisateur : référence de structure, non recopiée et non présentée comme source d'observations en Tunisie.
7. *UML Specification*, Object Management Group : documentation de référence pour les diagrammes de conception.
8. Documentation officielle Spring Boot, Angular, PostgreSQL, Flyway et Keycloak : à citer avec versions validées lors de l'implémentation.